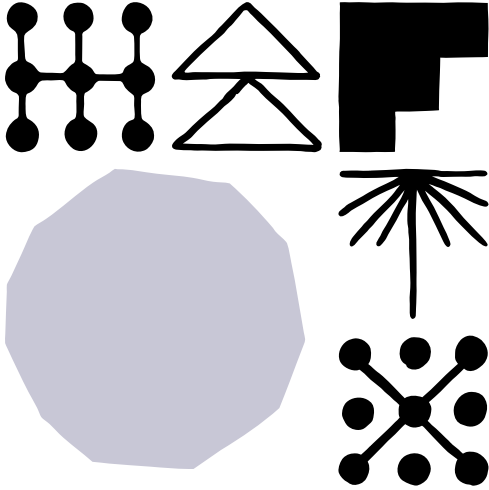


## Engineering at Anthropic

### Anthropic 的工程



# Effective context engineering for AI agents

## 面向 AI 代理的有效上下文工程

Context is a critical but finite resource for AI agents. In this post, we explore strategies for effectively curating and managing the context that powers them.

上下文是 AI 代理的关键。在这篇文章中，我们探讨了有效管理和维护驱动它们的上下文的方法。

---

Published Sep 29, 2025 2025 年 9 月 29 日发布

After a few years of prompt engineering being the focus of attention in applied AI, a new term has come to prominence: **context engineering**. Building with language models is becoming less about finding the right words and phrases for your prompts, and more about answering the broader question of “what configuration of

context is most likely to generate our model's desired behavior?"

在应用人工智能领域，提示工程成为关注焦点已有数年，一个新术语逐渐兴起：上下文工程。使用语言模型正在从寻找适合提示的正确词语和短语，转变为回答更广泛的问题：“哪种上下文配置最有可能生成我们模型期望的行为？”

**Context** refers to the set of tokens included when sampling from a large-language model (LLM). The **engineering** problem at hand is optimizing the utility of those tokens against the inherent constraints of LLMs in order to consistently achieve a desired outcome. Effectively wrangling LLMs often requires *thinking in context* — in other words: considering the holistic state available to the LLM at any given time and what potential behaviors that state might yield.

上下文指的是从大型语言模型（LLM）中采样时包含的标记集。当前面临的工程问题是优化这些标记的效用，以克服 LLM 的固有约束，从而持续实现期望的结果。有效管理 LLM 通常需要从上下文角度思考——换句话说：考虑在任何给定时间 LLM 可用的整体状态以及该状态可能产生的潜在行为。

In this post, we'll explore the emerging art of context engineering and offer a refined mental model for building steerable, effective agents.

在这篇文章中，我们将探讨新兴的上下文工程技术，并提供一个用于构建可控、高效智能体的精炼思维模型。

## Context engineering vs. prompt engineering

### 上下文工程与提示工程

**A**

三

engineering. Prompt engineering refers to methods for writing and organizing LLM instructions for optimal outcomes (see [our docs](#) for an overview and useful prompt engineering strategies). **Context engineering** refers to the set of strategies for curating and maintaining the optimal set of tokens (information) during LLM inference, including all the other information that may land there outside of the prompts.

在 Anthropic，我们将上下文工程视为提示工程的自然发展。提示工程是指为获得最佳结果而编写和组织 LLM 指令的方法（请参阅我们的文档了解概述和有用的提示工程策略）。上下文工程是指一组策略，用于在 LLM 推理过程中管理和维护最佳令牌集（信息），包括提示之外可能出现在其中的所有其他信息。

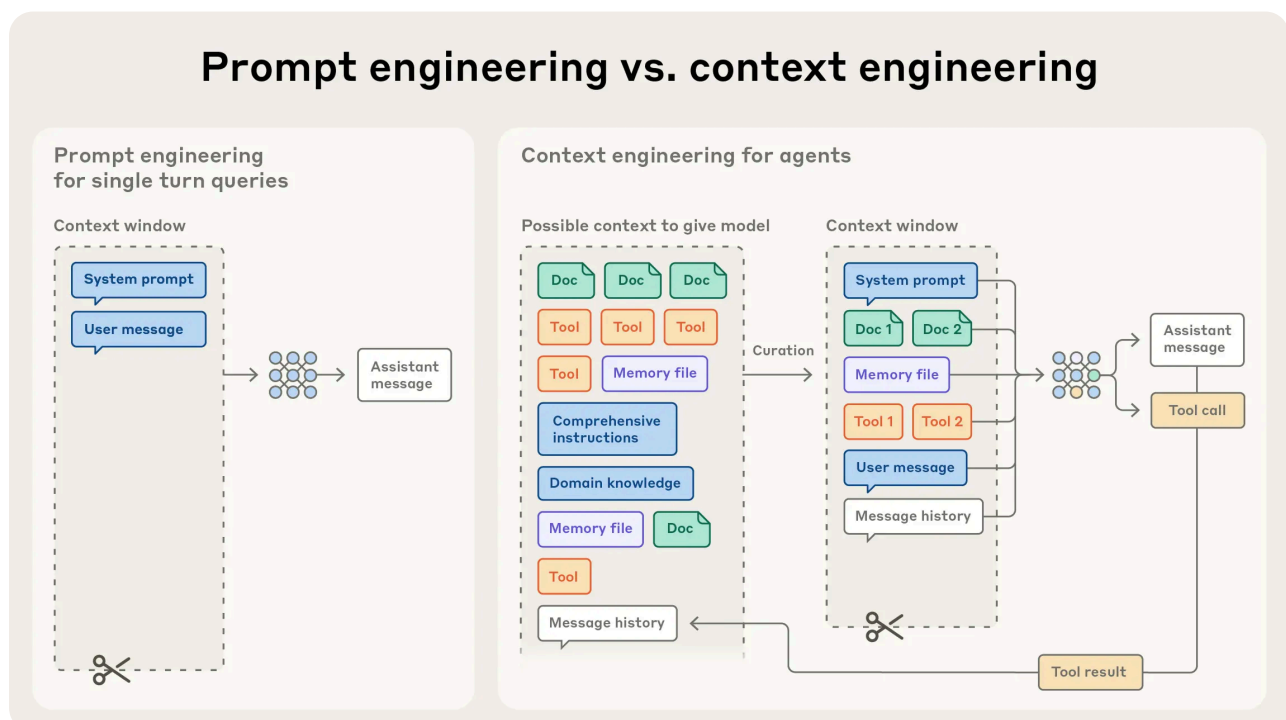
In the early days of engineering with LLMs, prompting was the biggest component of AI engineering work, as the majority of use cases outside of everyday chat

interactions required prompts optimized for one-shot classification or text generation tasks. As the term implies, the primary focus of prompt engineering is how to write effective prompts, particularly system prompts. However, as we move towards engineering more capable agents that operate over multiple turns of inference and longer time horizons, we need strategies for managing the entire context state (system instructions, tools, Model Context Protocol (MCP), external data, message history, etc).

在早期使用 LLM 进行工程设计的阶段，提示是 AI 工程设计工作的最大组成部分，因为在日常聊天交互之外的大多数用例需要针对单次分类或文本生成任务进行优化的提示。正如术语所暗示的，提示工程的主要重点是如何编写有效的提示，特别是系统提示。然而，随着我们转向设计和更强大的代理，这些代理能够在多轮推理和更长的时间范围内运行，我们需要管理整个上下文状态（系统指令、工具、模型上下文协议（MCP）、外部数据、消息历史等）的策略。

An agent running in a loop generates more and more data that *could* be relevant for the next turn of inference, and this information must be cyclically refined. Context engineering is the art and science of curating what will go into the limited context window from that constantly evolving universe of possible information.

一个在循环中运行的代理会生成越来越多可能与下一次推理相关的数据，这些信息必须被周期性地优化。上下文工程是管理和优化从不断发展的可能信息宇宙中选取有限上下文窗口内容的艺术和科学。



*In contrast to the discrete task of writing a prompt, context engineering is iterative and the curation phase happens each time we decide what to pass to the model.*

与编写提示的离散任务不同，上下文工程是迭代的，筛选阶段发生在我们每次决定向模型传递什么内容时。

## Why context engineering is important to building capable agents 为什么上下文工程对构建强大的代理很重要

Despite their speed and ability to manage larger and larger volumes of data, we've observed that LLMs, like humans, lose focus or experience confusion at a certain point. Studies on needle-in-a-haystack style benchmarking have uncovered the concept of context rot: as the number of tokens in the context window increases, the model's ability to accurately recall information from that context decreases.

尽管它们速度快，能够处理越来越大的数据量，但我们观察到，像人类一样，LLMs 在某个点上会失去焦点或感到困惑。关于针锋相对的基准测试研究揭示了上下文腐烂的概念：随着上下文窗口中令牌数量的增加，模型从该上下文中准确回忆信息的能力会下降。

While some models exhibit more gentle degradation than others, this characteristic emerges across all models. Context, therefore, must be treated as a finite resource with diminishing marginal returns. Like humans, who have limited working memory capacity, LLMs have an “attention budget” that they draw on when parsing large volumes of context. Every new token introduced depletes this budget by some amount, increasing the need to carefully curate the tokens available to the LLM.

尽管有些模型表现出比其他模型更温和的退化，但这种特性在所有模型中都存在。因此，上下文必须被视为一种具有递减边际收益的有限资源。就像人类拥有有限的 working memory 容量一样，LLMs 也有一个“注意力预算”，在解析大量上下文时会动用这个预算。引入的每个新 token 都会消耗一部分预算，从而增加了需要仔细管理 LLM 可用 token 的需求。

This attention scarcity stems from architectural constraints of LLMs. LLMs are based on the transformer architecture, which enables every token to attend to every other token across the entire context. This results in  $n^2$  pairwise relationships

for  $n$  tokens.

这种注意力稀缺源于 LLMs 的架构限制。LLMs 基于 Transformer 架构，该架构使每个 token 能够关注整个上下文中的每个其他 token。这导致  $n$  个 token 之间存在  $n^2$  对关系。

As its context length increases, a model's ability to capture these pairwise relationships gets stretched thin, creating a natural tension between context size and attention focus. Additionally, models develop their attention patterns from training data distributions where shorter sequences are typically more common than longer ones. This means models have less experience with, and fewer specialized parameters for, context-wide dependencies.

随着其上下文长度的增加，模型捕捉这些成对关系的能力会被逐渐削弱，从而在上下文大小和注意力焦点之间形成一种自然张力。此外，模型从训练数据分布中发展其注意力模式，而较短的序列通常比较长的序列更常见。这意味着模型在处理全局上下文依赖关系方面经验较少，并且专门用于这些依赖关系的参数也较少。

Techniques like position encoding interpolation allow models to handle longer sequences by adapting them to the originally trained smaller context, though with some degradation in token position understanding. These factors create a performance gradient rather than a hard cliff: models remain highly capable at longer contexts but may show reduced precision for information retrieval and long-range reasoning compared to their performance on shorter contexts.

像位置编码插值这样的技术允许模型通过将其适应最初训练的较小上下文来处理更长的序列，尽管在标记位置理解方面会有些许退化。这些因素形成了一种性能梯度，而不是一个陡峭的悬崖：模型在较长的上下文中仍然具有很强的能力，但在信息检索和长距离推理方面，其表现可能不如在较短上下文中的表现那样精确。

These realities mean that thoughtful context engineering is essential for building capable agents.

这些现实意味着，在构建强大的智能体时，需要精心设计上下文工程。

## The anatomy of effective context

### 有效上下文的结构

Given that LLMs are constrained by a finite attention budget, *good* context engineering means finding the *smallest possible* set of high-signal tokens that maximize the likelihood of some desired outcome. Implementing this practice is much easier said than done, but in the following section, we outline what this



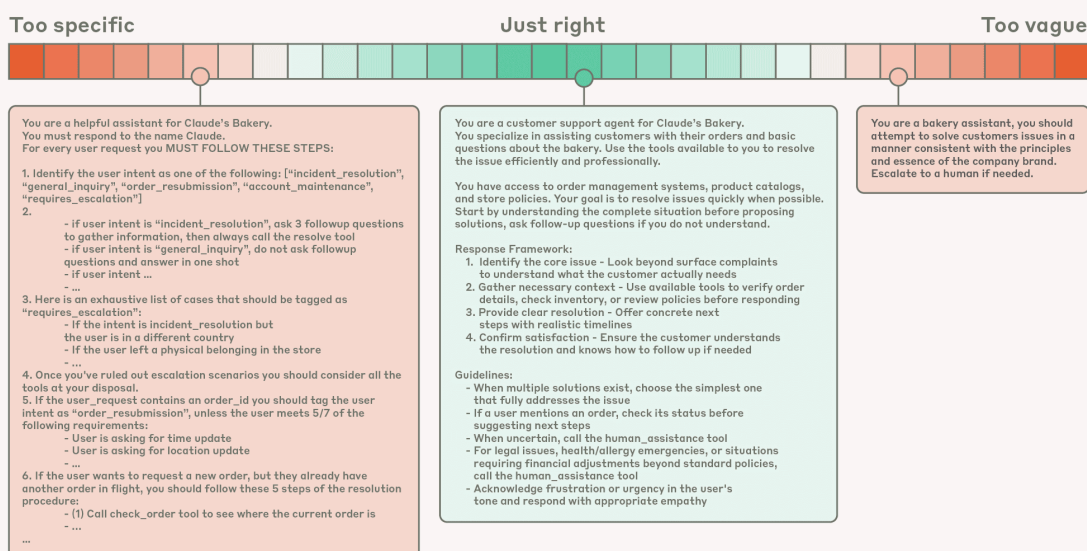
guiding principle means in practice across the different components of context.

鉴于 LLMs 受限于有限的注意力预算，良好的上下文工程意味着找到一组信号最强的最小标记集，以最大化实现某些预期结果的可能性。实施这一实践远比说起来容易得多，但在下一节中，我们概述了这一指导原则在实际中如何应用于上下文的不同组件。

**System prompts** should be extremely clear and use simple, direct language that presents ideas at the *right altitude* for the agent. The right altitude is the Goldilocks zone between two common failure modes. At one extreme, we see engineers hardcoding complex, brittle logic in their prompts to elicit exact agentic behavior. This approach creates fragility and increases maintenance complexity over time. At the other extreme, engineers sometimes provide vague, high-level guidance that fails to give the LLM concrete signals for desired outputs or falsely assumes shared context. The optimal altitude strikes a balance: specific enough to guide behavior effectively, yet flexible enough to provide the model with strong heuristics to guide behavior.

系统提示应当极其清晰，使用简单直接的语言，以恰当的高度向代理呈现思想。恰当的高度是两个常见故障模式之间的“黄金法则”区域。在一个极端，我们看到工程师在提示中硬编码复杂、脆弱的逻辑，以引出精确的代理行为。这种方法造成脆弱性，并随着时间的推移增加维护复杂性。在另一个极端，工程师有时提供模糊、高级的指导，未能向 LLM 提供关于预期输出的具体信号，或错误地假设共享上下文。最佳高度寻求平衡：具体到足以有效指导行为，同时又足够灵活，为模型提供强大的启发式方法来指导行为。

## Calibrating the system prompt



*At one end of the spectrum, we see brittle if-else hardcoded prompts, and at the other end we see prompts that are overly general or falsely assume shared context.*

在光谱的一端，我们看到脆弱的 *if-else* 硬编码提示，而在另一端，我们看到过于笼统或错误假设共享上下文的提示。

We recommend organizing prompts into distinct sections (like

`<background_information>`, `<instructions>`, `## Tool guidance`, `## Output description`, etc) and using techniques like XML tagging or Markdown headers to delineate these sections, although the exact formatting of prompts is likely becoming less important as models become more capable.

我们建议将提示组织成不同的部分（如 `<background_information>`、`<instructions>`、`## Tool guidance`、`## Output description` 等），并使用 XML 标签或 Markdown 标题等技术来划分这些部分，尽管随着模型能力的增强，提示的精确格式可能变得不那么重要。

Regardless of how you decide to structure your system prompt, you should be striving for the minimal set of information that fully outlines your expected behavior. (Note that minimal does not necessarily mean short; you still need to give the agent sufficient information up front to ensure it adheres to the desired behavior.) It's best to start by testing a minimal prompt with the best model available to see how it performs on your task, and then add clear instructions and examples to improve performance based on failure modes found during initial testing.

无论你怎么决定构建系统提示，都应努力实现充分说明预期行为的最少信息量。（注意：最少并不意味着简短；你仍然需要提前向代理提供足够的信息，以确保它遵循期望的行为。）最好从使用最佳模型测试最小提示开始，看看它在你的任务上表现如何，然后根据初始测试中发现的问题模式添加清晰的指令和示例来改进性能。

**Tools** allow agents to operate with their environment and pull in new, additional context as they work. Because tools define the contract between agents and their information/action space, it's extremely important that tools promote efficiency, both by returning information that is token efficient and by encouraging efficient agent behaviors.

工具使智能体能够与其环境交互，并在工作中引入新的附加上下文。由于工具定义了智能体与其信息/动作空间之间的契约，因此工具必须通过返回 **token** 高效的信息并鼓励高效的智能体行为来促进效率，这一点至关重要。

In Writing tools for AI agents - with AI agents, we discussed building tools that are well understood by LLMs and have minimal overlap in functionality. Similar to the functions of a well-designed codebase, tools should be self-contained, robust to error, and extremely clear with respect to their intended use. Input parameters should similarly be descriptive, unambiguous, and play to the inherent strengths of the model.

在《为 AI 智能体编写工具——使用 AI 智能体》中，我们讨论了构建被 LLMs 充分理解且功能重叠最小的工具。类似于精心设计的代码库的功能，工具应该是自包含的、能够容忍错误的，并且对其预期用途非常清晰。输入参数也应该具有描述性、无歧义，并发挥模型的固有优势。

One of the most common failure modes we see is bloated tool sets that cover too much functionality or lead to ambiguous decision points about which tool to use. If a human engineer can't definitively say which tool should be used in a given situation, an AI agent can't be expected to do better. As we'll discuss later, curating a minimal viable set of tools for the agent can also lead to more reliable maintenance and pruning of context over long interactions.

我们观察到最常见的失效模式之一是工具集过于臃肿，涵盖过多功能或导致在工具选择上出现模糊的决策点。如果人类工程师在特定情况下无法明确指出应使用哪个工具，那么就不应期望人工智能代理能做得更好。正如我们稍后将要讨论的，为代理精选一套最小可行工具集也能在长期交互中带来更可靠的维护和上下文剪枝。

Providing examples, otherwise known as few-shot prompting, is a well known best practice that we continue to strongly advise. However, teams will often stuff a laundry list of edge cases into a prompt in an attempt to articulate every possible rule the LLM should follow for a particular task. We do not recommend this. Instead, we recommend working to curate a set of diverse, canonical examples that effectively portray the expected behavior of the agent. For an LLM, examples are the “pictures” worth a thousand words.

提供示例，也称为少样本提示，是一种广为人知的最佳实践，我们持续强烈建议采用。然而，团队往往会将大量边缘案例塞进提示中，试图阐明 LLM 在特定任务中应遵循的所有可能规则。我们不推荐这种做法。相反，我们建议努力精选一套多样化、规范化的示例，这些示例能有效展现代理的预期行为。对于 LLM 而言，示例就是“千言万语”的“图画”。

Our overall guidance across the different components of context (system prompts, tools, examples, message history, etc) is to be thoughtful and keep your context informative, yet tight. Now let's dive into dynamically retrieving context at



runtime.

我们在上下文的不同组成部分（系统提示、工具、示例、消息历史等）的整体指导原则是深思熟虑，并保持上下文信息丰富且简洁。现在让我们深入探讨在运行时动态检索上下文。

## Context retrieval and agentic search

### 上下文检索与智能体搜索

In Building effective AI agents, we highlighted the differences between LLM-based workflows and agents. Since we wrote that post, we've gravitated towards a simple definition for agents: LLMs autonomously using tools in a loop.

在构建有效的 AI 智能体中，我们强调了基于 LLM 的工作流程与智能体之间的区别。自从我们撰写那篇文章以来，我们逐渐形成了一个简单的智能体定义：LLM 在循环中自主使用工具。

Working alongside our customers, we've seen the field converging on this simple paradigm. As the underlying models become more capable, the level of autonomy of agents can scale: smarter models allow agents to independently navigate nuanced problem spaces and recover from errors.

通过与客户合作，我们看到了该领域正趋近于这个简单的范式。随着底层模型能力的提升，智能体的自主性水平可以扩展：更智能的模型允许智能体独立地导航复杂的问题空间并从错误中恢复。

We're now seeing a shift in how engineers think about designing context for agents. Today, many AI-native applications employ some form of embedding-based pre-inference time retrieval to surface important context for the agent to reason over. As the field transitions to more agentic approaches, we increasingly see teams augmenting these retrieval systems with “just in time” context strategies.

我们现在看到工程师在设计智能体上下文时思维方式正在转变。如今，许多原生 AI 应用程序采用基于嵌入的预推理时间检索来呈现智能体推理的重要上下文。随着该领域向更具智能体特性的方法过渡，我们越来越多地看到团队通过“即时”上下文策略来增强这些检索系统。

Rather than pre-processing all relevant data up front, agents built with the “just in time” approach maintain lightweight identifiers (file paths, stored queries, web links, etc.) and use these references to dynamically load data into context at runtime using tools. Anthropic's agentic coding solution Claude Code uses this

approach to perform complex data analysis over large databases. The model can write targeted queries, store results, and leverage Bash commands like `head` and `tail` to analyze large volumes of data without ever loading the full data objects into context. This approach mirrors human cognition: we generally don't memorize entire corpuses of information, but rather introduce external organization and indexing systems like file systems, inboxes, and bookmarks to retrieve relevant information on demand.

与其在事前对所有相关数据进行预处理，采用“按需”方法的智能体则保持轻量级标识符（文件路径、存储查询、网页链接等），并利用这些引用在运行时通过工具动态将数据加载到上下文中。Anthropic 的智能体编程解决方案 Claude Code 采用这种方法来对大型数据库执行复杂的数据分析。该模型可以编写定向查询、存储结果，并利用 `head` 和 `tail` 等 Bash 命令来分析大量数据，而无需将完整的数据对象加载到上下文中。这种方法反映了人类认知：我们通常不会记忆整个信息库，而是引入外部组织索引系统（如文件系统、收件箱和书签）来按需检索相关信息。

Beyond storage efficiency, the metadata of these references provides a mechanism to efficiently refine behavior, whether explicitly provided or intuitive. To an agent operating in a file system, the presence of a file named `test_utils.py` in a `tests` folder implies a different purpose than a file with the same name located in `src/core_logic/`. Folder hierarchies, naming conventions, and timestamps all provide important signals that help both humans and agents understand how and when to utilize information.

除了存储效率之外，这些引用的元数据还提供了一种有效优化行为的机制，无论是明确提供的还是直觉性的。对于一个在文件系统中运行的代理来说，一个名为 `test_utils.py` 的文件存在于 `tests` 文件夹中，其含义不同于一个具有相同名称的文件位于 `src/core_logic/` 文件夹层次结构中。命名规范、时间戳等所有这些信号都为人类和代理提供了重要信息，帮助他们理解何时以及如何利用信息。

Letting agents navigate and retrieve data autonomously also enables progressive disclosure—in other words, allows agents to incrementally discover relevant context through exploration. Each interaction yields context that informs the next decision: file sizes suggest complexity; naming conventions hint at purpose; timestamps can be a proxy for relevance. Agents can assemble understanding layer by layer, maintaining only what's necessary in working memory and leveraging note-taking strategies for additional persistence. This self-managed context window keeps the agent focused on relevant subsets rather than drowning

in exhaustive but potentially irrelevant information.

让代理自主导航和检索数据，也实现了渐进式披露——换句话说，允许代理通过探索逐步发现相关上下文。每次交互都会产生用于指导下一次决策的上下文：文件大小暗示复杂性；命名规范暗示用途；时间戳可以作为相关性的代理。代理可以逐层构建理解，仅在工作记忆中保留必要信息，并利用笔记策略实现额外持久化。这种自我管理的上下文窗口使代理专注于相关子集，而不是被大量但可能无关的信息淹没。

Of course, there's a trade-off: runtime exploration is slower than retrieving pre-computed data. Not only that, but opinionated and thoughtful engineering is required to ensure that an LLM has the right tools and heuristics for effectively navigating its information landscape. Without proper guidance, an agent can waste context by misusing tools, chasing dead-ends, or failing to identify key information.

当然，存在权衡：运行时探索比检索预计算数据更慢。不仅如此，还需要有见地和深思熟虑的工程实践，以确保 LLM 拥有正确工具和启发式方法来有效导航其信息景观。如果没有适当的指导，代理可能会因误用工具、追逐死胡同或未能识别关键信息而浪费上下文。

In certain settings, the most effective agents might employ a hybrid strategy, retrieving some data up front for speed, and pursuing further autonomous exploration at its discretion. The decision boundary for the 'right' level of autonomy depends on the task. Claude Code is an agent that employs this hybrid model: CLAUDE.md files are naively dropped into context up front, while primitives like glob and grep allow it to navigate its environment and retrieve files just-in-time, effectively bypassing the issues of stale indexing and complex syntax trees.

在某些场景下，最有效的智能体可能会采用混合策略，在开始时检索部分数据以提升速度，并在其自主判断下进行进一步的自主探索。决定“恰当”自主程度的决策边界取决于具体任务。Claude Code 是一个采用这种混合模型的智能体：CLAUDE.md 文件被直接置入上下文中，而 glob 和 grep 等基础操作则允许它导航其环境并即时检索文件，有效绕过了陈旧索引和复杂语法树的问题。

The hybrid strategy might be better suited for contexts with less dynamic content, such as legal or finance work. As model capabilities improve, agentic design will trend towards letting intelligent models act intelligently, with progressively less human curation. Given the rapid pace of progress in the field, "do the simplest thing that works" will likely remain our best advice for teams building agents on

top of Claude.

混合策略可能更适合内容变化较少的上下文，例如法律或金融工作。随着模型能力的提升，代理设计将倾向于让智能模型智能地行动，人类干预将越来越少。鉴于该领域的快速发展，“做最简单有效的事”很可能仍然是我们在基于 Claude 构建代理时最好的建议。

## Context engineering for long-horizon tasks

### 长期任务中的上下文工程

Long-horizon tasks require agents to maintain coherence, context, and goal-directed behavior over sequences of actions where the token count exceeds the LLM's context window. For tasks that span tens of minutes to multiple hours of continuous work, like large codebase migrations or comprehensive research projects, agents require specialized techniques to work around the context window size limitation.

长期任务需要代理在一系列操作中保持连贯性、上下文和目标导向行为，而操作数量超过了 LLM 的上下文窗口。对于需要持续工作数分钟到数小时的任务，如大型代码库迁移或全面研究项目，代理需要专门的技巧来规避上下文窗口大小限制。

Waiting for larger context windows might seem like an obvious tactic. But it's likely that for the foreseeable future, context windows of all sizes will be subject to context pollution and information relevance concerns—at least for situations where the strongest agent performance is desired. To enable agents to work effectively across extended time horizons, we've developed a few techniques that address these context pollution constraints directly: compaction, structured note-taking, and multi-agent architectures.

等待更大的上下文窗口可能看似是一种明显的策略。但可以预见的是，在可预见的未来，所有大小的上下文窗口都将受到上下文污染和信息相关性问题的困扰——至少在需要最强代理性能的情况下。为了使代理能够在扩展的时间范围内有效工作，我们开发了几种直接解决这些上下文污染约束的技术：压缩、结构化笔记和多层代理架构。

### Compaction 压缩

Compaction is the practice of taking a conversation nearing the context window limit, summarizing its contents, and reinitiating a new context window with the summary. Compaction typically serves as the first lever in context engineering to drive better long-term coherence. At its core, compaction distills the contents of a context window in a high-fidelity manner, enabling the agent to continue with



minimal performance degradation.

压缩是指对接近上下文窗口限制的对话进行内容总结，并重新初始化一个新的上下文窗口。压缩通常是上下文工程中的第一个杠杆，用于提升长期连贯性。其核心在于以高保真方式提炼上下文窗口的内容，使智能体能够以最小的性能下降继续执行。

In Claude Code, for example, we implement this by passing the message history to the model to summarize and compress the most critical details. The model preserves architectural decisions, unresolved bugs, and implementation details while discarding redundant tool outputs or messages. The agent can then continue with this compressed context plus the five most recently accessed files. Users get continuity without worrying about context window limitations.

在 Claude Code 中，例如，我们通过将消息历史传递给模型来实施这一方法，以总结和压缩最关键的信息。模型保留了架构决策、未解决的错误和实现细节，同时丢弃了冗余的工具输出或消息。然后，代理可以继续使用这个压缩后的上下文以及最近访问的前五个文件。用户无需担心上下文窗口的限制，从而获得连续性。

The art of compaction lies in the selection of what to keep versus what to discard, as overly aggressive compaction can result in the loss of subtle but critical context whose importance only becomes apparent later. For engineers implementing compaction systems, we recommend carefully tuning your prompt on complex agent traces. Start by maximizing recall to ensure your compaction prompt captures every relevant piece of information from the trace, then iterate to improve precision by eliminating superfluous content.

压缩的艺术在于选择保留什么与丢弃什么，因为过于激进的压缩可能导致后来才显现重要性的微妙但关键的上下文信息丢失。对于实施压缩系统的工程师，我们建议在复杂的智能体轨迹上仔细调整你的提示。首先通过最大化召回率来确保你的压缩提示从轨迹中捕获所有相关信息，然后通过迭代消除冗余内容来提高精确度。

An example of low-hanging superfluous content is clearing tool calls and results – once a tool has been called deep in the message history, why would the agent need to see the raw result again? One of the safest lightest touch forms of compaction is tool result clearing, most recently launched as a feature on the Claude Developer Platform.

一个低垂的冗余内容示例是清除工具调用和结果——一旦一个工具在消息历史中深处被调用，代理为什么还需要再次看到原始结果？工具结果清除是最安全、最轻量级压缩形式之一，最近作为功能在 Claude 开发者平台上推出。

## Structured note-taking 结构化笔记

Structured note-taking, or agentic memory, is a technique where the agent regularly writes notes persisted to memory outside of the context window. These notes get pulled back into the context window at later times.

结构化笔记，或称为代理式记忆，是一种技术，即代理定期将笔记写入到上下文窗口之外的持久化记忆中。这些笔记在稍后时间会被重新拉回到上下文窗口中。

This strategy provides persistent memory with minimal overhead. Like Claude Code creating a to-do list, or your custom agent maintaining a NOTES.md file, this simple pattern allows the agent to track progress across complex tasks, maintaining critical context and dependencies that would otherwise be lost across dozens of tool calls.

这种策略提供了具有最小开销的持久化记忆。就像 Claude Code 创建待办事项列表，或你的自定义代理维护 NOTES.md 文件一样，这种简单的模式允许代理跟踪复杂任务的进度，保持关键上下文和依赖关系，而这些在数十次工具调用中可能会丢失。

Claude playing Pokémon demonstrates how memory transforms agent capabilities in non-coding domains. The agent maintains precise tallies across thousands of game steps—tracking objectives like "for the last 1,234 steps I've been training my Pokémon in Route 1, Pikachu has gained 8 levels toward the target of 10." Without any prompting about memory structure, it develops maps of explored regions, remembers which key achievements it has unlocked, and maintains strategic notes of combat strategies that help it learn which attacks work best against different opponents.

Claude 玩宝可梦展示了记忆如何改变非编码领域的代理能力。代理在数千个游戏步骤中保持精确的计数——跟踪目标，例如“在最后 1,234 步中，我在 1 号路训练宝可梦，皮卡丘已经获得了 8 级，朝着 10 级的目标前进。”没有任何关于记忆结构的提示，它开发了探索地区的地图，记得已经解锁了哪些关键成就，并保持战斗策略的战术笔记，这些笔记帮助它学习哪些攻击对不同对手最有效。

After context resets, the agent reads its own notes and continues multi-hour training sequences or dungeon explorations. This coherence across summarization steps enables long-horizon strategies that would be impossible when keeping all the information in the LLM's context window alone.

在上下文重置后，智能体读取自己的笔记，并继续进行多小时的训练序列或地牢探索。这种在摘要步骤之间的连贯性使得能够实现长时程策略，而这些策略如果仅将所有信息保留在 LLM 的上下文窗口中则是不可能的。

As part of our Sonnet 4.5 launch, we released a memory tool in public beta on the Claude Developer Platform that makes it easier to store and consult information outside the context window through a file-based system. This allows agents to build up knowledge bases over time, maintain project state across sessions, and reference previous work without keeping everything in context.

作为我们 Sonnet 4.5 发布的一部分，我们在 Claude 开发者平台上发布了内存工具的公开测试版，该工具通过基于文件的系统使存储和查询上下文窗口外部的信息变得更加容易。这允许智能体随着时间的推移建立知识库，跨会话维护项目状态，并在无需将所有内容保留在上下文中即可引用之前的工作。

## Sub-agent architectures 子智能体架构

Sub-agent architectures provide another way around context limitations. Rather than one agent attempting to maintain state across an entire project, specialized sub-agents can handle focused tasks with clean context windows. The main agent coordinates with a high-level plan while subagents perform deep technical work or use tools to find relevant information. Each subagent might explore extensively, using tens of thousands of tokens or more, but returns only a condensed, distilled summary of its work (often 1,000-2,000 tokens).

子代理架构为解决上下文限制提供了另一种方法。与其让单个代理试图在整个项目中维持状态，专门的子代理可以处理聚焦的任务，并保持清晰的上下文窗口。主代理负责与高层计划协调，而子代理则执行深度的技术工作或使用工具查找相关信息。每个子代理可能会进行广泛的探索，使用数万甚至更多的 token，但只返回其工作的精炼摘要（通常为 1,000-2,000 个 token）。

This approach achieves a clear separation of concerns—the detailed search context remains isolated within sub-agents, while the lead agent focuses on synthesizing and analyzing the results. This pattern, discussed in How we built our multi-agent research system, showed a substantial improvement over single-agent systems on complex research tasks.

这种方法实现了明确的关注点分离——详细搜索上下文保持隔离在子代理中，而主代理则专注于综合和分析结果。这种模式，如我们在《如何构建我们的多代理研究系统》中讨论的，在复杂研究任务上显示出比单代理系统大幅的改进。

The choice between these approaches depends on task characteristics. For example:

这些方法的选择取决于任务特征。例如：

- **Compaction maintains conversational flow for tasks requiring extensive back-and-forth;**  
压缩（Compaction）维持需要大量来回交流的任务的对话流程；
- **Note-taking excels for iterative development with clear milestones;**  
做笔记非常适合具有明确里程碑的迭代开发；
- **Multi-agent architectures handle complex research and analysis where parallel exploration pays dividends.**  
多智能体架构能够处理复杂的研究与分析，其中并行探索能带来收益。

Even as models continue to improve, the challenge of maintaining coherence across extended interactions will remain central to building more effective agents. 即使模型持续改进，保持跨长时间交互的连贯性仍将是构建更有效智能体的核心挑战。

## Conclusion 结论

Context engineering represents a fundamental shift in how we build with LLMs. As models become more capable, the challenge isn't just crafting the perfect prompt—it's thoughtfully curating what information enters the model's limited attention budget at each step. Whether you're implementing compaction for long-horizon tasks, designing token-efficient tools, or enabling agents to explore their environment just-in-time, the guiding principle remains the same: find the smallest set of high-signal tokens that maximize the likelihood of your desired outcome.

上下文工程代表着我们使用 LLMs 构建方式的根本性转变。随着模型能力的提升，挑战不仅在于设计完美的提示——更在于精心管理每一步中进入模型有限注意力预算的信息。无论是为长时程任务实现压缩，设计高效的工具，还是让智能体即时探索环境，指导原则始终不变：找到能最大化预期结果可能性的最小高信号标记集。

The techniques we've outlined will continue evolving as models improve. We're already seeing that smarter models require less prescriptive engineering, allowing agents to operate with more autonomy. But even as capabilities scale, treating context as a precious, finite resource will remain central to building reliable, effective agents.

我们概述的技术将随着模型的改进而持续发展。我们已经看到更智能的模型需要更少的规定性工程，允许智能体以更多自主性运行。但即使能力扩展，将上下文视为珍贵且有限的资源，对于构建可靠有效的智能体仍将是核心。



Get started with context engineering in the Claude Developer Platform today, and access helpful tips and best practices via our [memory and context management cookbook](#).

从今天开始在 Claude 开发者平台上进行上下文工程，并通过我们的记忆和上下文管理烹饪书获取有用的技巧和最佳实践。

## Acknowledgements 致谢

Written by Anthropic's Applied AI team: Prithvi Rajasekaran, Ethan Dixon, Carly Ryan, and Jeremy Hadfield, with contributions from team members Rafi Ayub, Hannah Moran, Cal Rueb, and Connor Jennings. Special thanks to Molly Vorwerck, Stuart Ritchie, and Maggie Vo for their support.

由 Anthropic 的应用人工智能团队撰写：Prithvi Rajasekaran、Ethan Dixon、Carly Ryan 和 Jeremy Hadfield，团队成员 Rafi Ayub、Hannah Moran、Cal Rueb 和 Connor Jennings 做出贡献。特别感谢 Molly Vorwerck、Stuart Ritchie 和 Maggie Vo 的支持。

## Get the developer newsletter

### 获取开发者简报

Product updates, how-tos, community spotlights, and more.  
Delivered monthly to your inbox.

产品更新、使用指南、社区聚焦等内容。每月发送至您的邮箱。

Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.

如需接收我们的每月开发者简报，请提供您的邮箱地址。您可以随时取消订阅。



## Products

Claude

Claude Code

Cowork

Claude in Chrome

Claude in Excel

Claude in PowerPoint

Claude in Slack

Skills

Max plan

Team plan

Enterprise plan

Download app

Pricing

Log in to Claude

## **Models**

Opus

Sonnet

Haiku

## **Solutions**

AI agents

Code modernization

Coding

Customer support

Education

Financial services

Government

Healthcare

Life sciences

Nonprofits

## Claude Developer Platform

Overview

Developer docs

Pricing

Regional compliance

Amazon Bedrock

Google Cloud's Vertex AI

Console login

## Learn

Blog

Claude partner network

Connectors

Courses

Customer stories

Engineering at Anthropic

Events

Plugins

Powered by Claude

Service partners

Startups program

Tutorials

Use cases

## Company

[Anthropic](#)

[Careers](#)

[Economic Futures](#)

[Research](#)

[News](#)

[Claude's Constitution](#)

[Responsible Scaling Policy](#)

[Security and compliance](#)

[Transparency](#)

## Help and security

[Availability](#)

[Status](#)

[Support center](#)

## Terms and policies

[Privacy choices](#)

[Privacy policy](#)

[Consumer health data privacy policy](#)

[Responsible disclosure policy](#)

[Terms of service: Commercial](#)

[Terms of service: Consumer](#)

[Usage policy](#)

© 2026 Anthropic PBC





